

목차

1장 확인의 목적	2
1.1 무엇을 확인하는가	2
1.2 확인이 주는 것	3
1.3 확인의 층	4
1.4 통과가 뜻하는 것	5
1.5 무엇을 확인하지 않을지	6
1.6 확인의 층과 범위	7
1.7 확인을 언제 돌리는가	8
2장 단위 확인	9
2.1 단위 확인의 정의	9
2.2 무엇을 한 단위로 볼까	10
2.3 확인의 구조	11
2.4 하나의 확인에 하나의 물음	12
2.5 확인 하나의 구조	13
2.6 이름이 알려 주는 것	14
2.7 깨지기 쉬운 확인	15
3장 대체물	16
3.1 대체물의 정의	16
3.2 대체물의 종류	17
3.3 무엇을 대체할까	18
3.4 대체물이 주는 착각	19
3.5 대체물이 놓이는 자리	20
3.6 대체물을 줄이는 설계	21
3.7 대체물이 낡을 때	22
4장 통합 확인과 계약	23
4.1 통합 확인의 자리	23
4.2 계약 검증	24
4.3 양쪽에서 같은 계약 보기	25
4.4 계약을 사이에 둔 두 확인	26
4.5 통합 확인의 비용	27
4.6 무엇까지 붙여 볼까	28
4.7 바깥 시스템과의 확인	29
5장 회귀 방지	30
5.1 회귀의 뜻	30
5.2 버그에서 확인으로	31
5.3 확인이 쌓이는 방식	32
5.4 느려지는 확인	33
5.5 흔들리는 확인	34
5.6 확인을 지우는 기준	35
5.7 확인이 알려 주는 설계	36
6장 한 기능을 확인으로 감싼 기록	37
6.1 감쌀 기능과 상태	37
6.2 기대를 먼저 적기	38
6.3 층을 나눈 기준	39
6.4 대체물을 만든 방식	40
6.5 통합 확인의 준비	41
6.6 확인이 드러낸 것	42
6.7 원래 하려던 변경	43
7장 확인 전략 판단	44
7.1 얼마나 확인할 것인가	44
7.2 통과를 어떻게 읽을까	45
7.3 층을 다시 배분할 때	46
7.4 확인 전략 점검 항목	47
7.5 확인을 고치는 순서	48
7.6 확인과 다른 안전 장치	49
7.7 핵심 원칙 정리	50

1.1 무엇을 확인하는가

확인 은 코드가 기대한 대로 동작하는지 보는 일이다. 여기서 기대는 사람의 머릿속이 아니라 어딘가에 적힌 것이어야 한다. 적히지 않은 기대는 확인할 수 없고, 확인할 수 없으면 코드가 바뀌었을 때 무엇이 깨졌는지 말할 수 없다.

그래서 확인을 짜는 일은 기대를 적는 일에서 시작한다. 어떤 조건에서 무엇이 나와야 하는지를 적고, 그 문장을 실행할 수 있는 형태로 옮긴다. 순서를 뒤집어 코드를 먼저 보고 확인을 짜면, 지금 동작을 그대로 옮겨 적게 되어 잘못된 동작까지 지켜 준다.

확인이 겨누지 않는 것도 분명하다. 코드가 어떻게 짜였는지는 확인의 대상이 아니다. 내부를 바꿔도 기대한 결과가 같으면 확인은 통과해야 한다. 이 구분이 서 있지 않으면 고칠 때마다 확인이 함께 깨져 확인이 짐이 된다.

1.2 확인이 주는 것

확인이 주는 첫째는 바꿀 수 있는 상태다. 고친 뒤 무엇이 깨졌는지 곧바로 알 수 있으면 손대는 범위를 넓게 잡을 수 있다. 확인이 없으면 잘 도는 코드를 건드리지 않는 쪽을 고르게 되고, 그렇게 굳은 코드는 시간이 지날수록 고치기 어려워진다.

둘째는 적힌 기대다. 확인은 이 코드가 무엇을 하기로 되어 있는지 실행 가능한 형태로 남긴다. 문서와 달리 어긋나면 곧바로 드러나므로 낱지 않는다.

셋째는 설계에 대한 되먹임이다. 확인을 짜기 어려운 코드는 대개 의존이 많거나 책임이 섞여 있다. 확인이 잘 짜이지 않을 때 확인을 억지로 맞추는 대신 코드를 다시 보는 편이 나은 경우가 많다. 이 셋이 이 책이 확인에 기대하는 전부다.

1.3 확인의 층

확인에는 겨누는 범위에 따라 층으로 나뉜다. 함수나 모듈 하나만 보는 확인, 여러 부분을 붙여 보는 확인, 시스템 전체를 바깥에서 부르는 확인이다. 이 책은 앞의 둘을 주로 다룬다.

층을 나누는 이유는 얻는 것과 드는 비용이 다르기 때문이다. 좁은 확인은 빠르고 원인을 정확히 가리키지만 부분들이 함께 동작하는지는 말해 주지 않는다. 넓은 확인은 실제에 가깝지만 느리고, 깨졌을 때 어디가 원인인지 좁혀 주지 않는다.

그래서 한 층으로 전부를 하려 하면 어느 쪽이든 손해가 난다. 좁은 확인만 있으면 붙였을 때 어긋나고, 넓은 확인만 있으면 느려서 자주 돌리지 못한다. 무엇을 어느 층에서 확인할지 정하는 일이 이 책이 말하는 전략이다.

1.4 통과가 뜻하는 것

확인이 모두 통과했다는 말은 적어 둔 기대가 지켜졌다는 뜻이지 코드에 문제가 없다는 뜻이 아니다. 적지 않은 기대는 확인되지 않았고, 생각하지 못한 조건은 애초에 확인 대상이 아니었다.

그래서 통과를 안전으로 읽으면 위험하다. 특히 확인의 수가 많을수록 이 착각이 커진다. 수가 많다는 것은 확인한 항목이 많다는 뜻이지 빠진 항목이 적다는 뜻이 아니다. 세어야 할 것은 확인의 수가 아니라 어떤 기대가 확인되고 있는가다.

통과를 읽는 올바른 방식은 이렇다. 지금까지 알고 있는 기대는 지켜지고 있다. 새 문제가 나오면 그 문제는 아직 적지 않은 기대였다는 뜻이므로, 고치기 전에 그 기대를 확인으로 옮긴다. 확인의 목록은 겪은 문제의 목록이기도 하다.

1.5 무엇을 확인하지 않을지

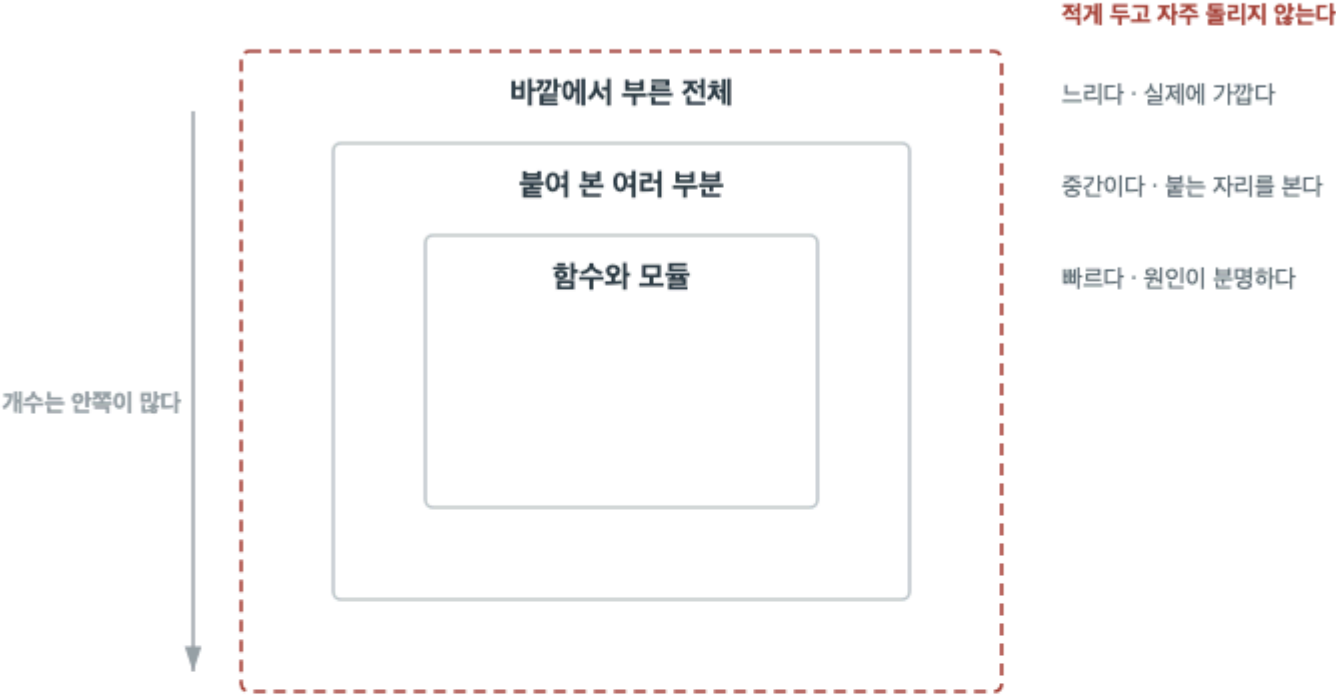
모든 것을 확인하려 하면 확인을 유지하는 일이 코드를 고치는 일보다 커진다. 그래서 확인하지 않을 것을 정한다.

빠기 좋은 것은 셋이다. 바깥 도구가 이미 보장하는 동작, 값을 그대로 옮기기만 하는 코드, 그리고 자주 바뀌는 걸모습이다. 앞의 둘은 확인해도 새로 알려 주는 것이 없고, 셋째는 확인이 곧바로 낡는다.

반대로 반드시 확인할 것도 정해 둔다. 조건에 따라 갈라지는 판단, 값의 경계, 돈이나 권한이 걸린 처리다. 이 둘을 미리 정해 두면 무엇을 확인할지 매번 고민하지 않아도 되고, 확인의 수가 아니라 확인의 대상으로 이야기할 수 있게 된다. 빠기로 한 목록도 적어 두어야 다음 사람이 누락으로 읽지 않는다.

확인의 총과 범위

넓을수록 실제에 가깝고 좁을수록 원인을 가리킨다



총마다 말을 물음을 다르게 준다

1.7 확인을 언제 돌리는가

확인으 돌려야 값을 한다. 언제 돌리느냐에 따라 같은 확인이 다른 쓸모를 갖는다. 짜는 동안 자주 돌리는 확인은 방금 한 변경을 알려 주고, 합치기 전에 돌리는 확인은 남의 변경과 부딪히는지 알려 주며, 배포 전에 돌리는 확인은 지금 나가는 것이 괜찮은지 알려 준다.

그래서 총과 시점을 짝지어 둔다. 빠른 확인은 자주 돌리고 느린 확인은 합치기 전이나 배포 전에 돌린다. 모든 확인을 매번 돌리려 하면 기다리는 시간 때문에 결국 아무도 돌리지 않는다.

돌리지 않는 확인은 없는 것과 같다. 오히려 나쁘다. 있다고 믿게 만들면서 실제로는 지키는 것이 없기 때문이다. 그래서 확인을 더할 때는 그 확인이 언제 돌 것인지도 함께 정한다.

2.1 단위 확인의 정의

단위 확인은 코드의 한 부분만 떼어 그 부분이 기대대로 동작하는지 보는 확인이다. 떼어 낸다는 말은 그 부분이 기대는 다른 것들을 실제로 부르지 않는다는 뜻이다.

떼어 내는 이유는 두 가지다. 빠르기 때문이고, 깨졌을 때 원인이 그 부분에 있음이 분명하기 때문이다. 여러 부분이 함께 도는 확인은 깨졌을 때 어느 부분 때문인지 다시 좁혀야 한다.

단위 확인이 잘 짜이려면 그 부분이 떼어 낼 수 있게 짜여 있어야 한다. 값을 인자로 받고 결과를 돌려주는 코드는 쉽게 떼어지고, 안에서 바깥을 직접 부르는 코드는 떼기 어렵다. 그래서 단위 확인의 난이도는 그 코드의 구조를 그대로 비춘다. 떼어 내기 어렵다는 사실 자체가 그 코드에 대한 정보다.

2.2 무엇을 한 단위로 볼까

단위를 함수 하나로 잡을지, 여러 함수가 모인 덩어리로 잡을지는 정해진 답이 없다. 기준은 그 단위가 기대를 말할 수 있는 크기인가다. 기대를 문장으로 적을 수 없으면 단위가 너무 작거나 크다.

너무 작게 잡으면 확인이 구현을 그대로 옮겨 적게 된다. 내부 함수 하나하나를 확인하면 그 함수를 합치거나 이름을 바꿀 때마다 확인이 깨진다. 이런 확인은 코드를 고치기 어렵게 만든다.

너무 크게 잡으면 원인을 가리키지 못한다. 실용적인 크기는 밖에서 부를 수 있는 자리, 곧 다른 코드가 실제로 쓰는 인터페이스 단위다. 그 자리를 단위로 잡으면 안쪽을 바꿔도 확인이 살아남고, 깨졌을 때 범위도 좁다. 단위가 인터페이스와 같으면 확인이 곧 그 인터페이스의 사용 예가 된다.

2.3 확인의 구조

확인 하나는 세 부분으로 이루어진다. 조건을 갖추는 부분, 확인할 동작을 실행하는 부분, 결과가 기대와 같은지 보는 부분이다. 이 셋이 눈에 보이게 나뉘어 있으면 읽는 사람이 무엇을 확인하는지 곧바로 안다.

셋이 섞이면 읽기 어려워진다. 조건을 갖추는 코드가 길어져 실행 부분이 묻히거나, 결과를 보는 부분이 여러 곳에 흩어지면 이 확인이 무엇을 지켜 주는지 알기 어렵다.

조건을 갖추는 부분이 너무 길면 그 자체가 신호다. 확인하려는 코드가 준비할 것이 많다는 뜻이고, 대개 의존이 많다는 뜻이다. 이때는 확인을 다듬기 전에 코드를 다시 보는 편이 낫다. 세 부분의 경계를 빈 줄 하나로만 나뉘어도 읽는 속도가 달라진다.

2.4 하나의 확인에 하나의 물음

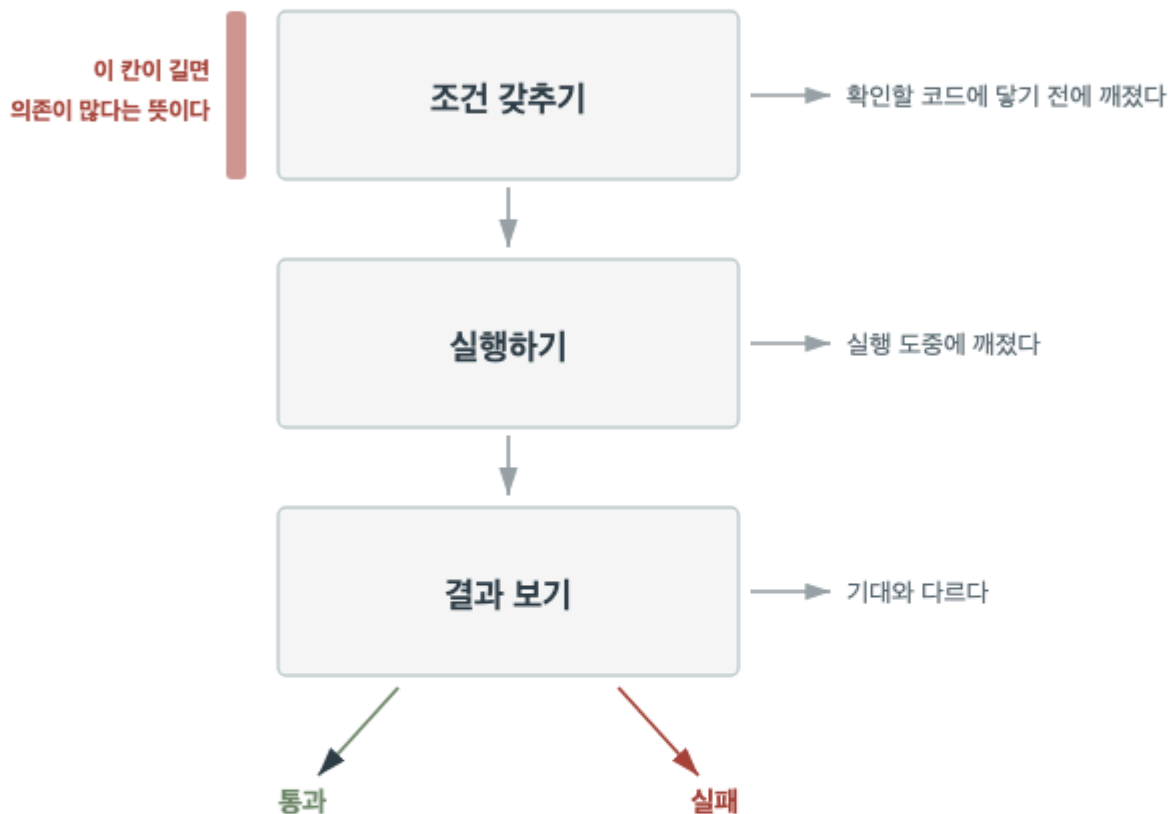
확인 하나에 여러 물음을 담으면 깨졌을 때 어느 물음이 깨진 것인지 알기 어렵다. 앞의 물음에서 실패하면 뒤의 물음은 확인조차 되지 않으므로, 고치고 다시 돌릴 때마다 새 실패가 하나씩 드러난다.

그래서 물음 하나에 확인 하나를 둔다. 같은 조건을 여러 확인이 함께 쓴다면 그 조건을 갖추는 코드를 함께 쓰면 되고, 확인 자체를 합칠 이유는 없다.

다만 물음 하나가 결과 여러 개를 함께 보는 것은 다른 이야기다. 한 처리의 결과로 값이 셋 바뀌어야 한다면 셋을 함께 보는 것이 그 물음이다. 나누는 기준은 확인하는 문장의 수가 아니라 물음의 수다. 물음이 하나면 실패 목록만 보고도 무엇이 깨졌는지 알 수 있다.

확인 하나의 구조

어디서 멈췄는지가 곧 무엇이 깨졌는지도



세 칸이 나뉘어 있어야 실패를 읽을 수 있다

2.6 이름이 알려 주는 것

확인 이름은 실패 목록에서 가장 먼저 읽히는 문장이다. 실패했을 때 이름만 보고 무엇이 깨졌는지 알 수 있으면 코드를 열기 전에 범위가 좁혀진다.

그래서 이름에는 조건과 기대를 담는다. 어떤 상황에서 무엇이 되어야 하는지가 들어가면 그 자체로 기대의 목록이 된다. 반대로 함수 이름을 그대로 옮긴 이름은 아무것도 알려 주지 않는다.

이름을 잘 짓다 보면 확인이 무엇을 겨누는지 스스로 정리된다. 이름을 짓기 어려운 확인은 대개 물음이 여럿이거나 겨누는 것이 분명하지 않은 확인이다. 이름은 결과물이 아니라 확인을 다듬는 도구다. 이름은 확인을 짠 사람이 아니라 실패를 보는 사람을 위해 짓는다. 이름만 모아 읽어도 이 코드가 무엇을 약속하는지 보인다.

2.7 깨지기 쉬운 확인

고칠 때마다 함께 깨지는 확인이 있다. 기대가 바뀌지 않았는데도 깨진다면 그 확인은 기대가 아니라 구현에 매여 있는 것이다.

흔한 원인은 셋이다. 내부 함수가 몇 번 불렸는지를 확인하거나, 결과의 순서처럼 약속한 적 없는 성질에 기대거나, 준비 단계에서 내부 구조를 그대로 만들어 넣는 경우다. 셋 다 안쪽을 바꾸면 깨진다.

이런 확인이 쌓이면 사람들은 코드를 고칠 때마다 확인을 함께 고치고, 그러다 확인을 지우거나 무시하게 된다. 확인이 변경을 돕는 것이 아니라 막게 되는 것이다. 확인이 자주 깨지면 확인을 고치기 전에 무엇에 매여 있는지 먼저 본다. 깨지는 확인을 고치기 전에 그 확인이 무엇을 지켜 주는지 먼저 묻는다.

3.1 대체물의 정의

대체물은 확인할 때 진짜 구성 요소 대신 놓는 물건이다. 실제 저장소나 바깥 시스템을 부르는 대신 미리 정한 값을 돌려주는 것을 두어 확인을 빠르고 예측 가능하게 만든다.

대체물이 필요한 이유는 셋이다. 진짜가 느리거나, 준비하기 어렵거나, 원하는 상황을 만들 수 없기 때문이다. 상대가 실패하는 경우를 확인하려면 실제로 실패시키기보다 실패를 돌려주는 대체물을 두는 편이 쉽다.

대체물의 값은 상황을 마음대로 만들 수 있다는 데 있고, 위험도 거기서 온다. 마음대로 만든 상황이 실제와 다르면 확인은 실제로 일어나지 않는 일을 지켜 준다. 이 장은 그 위험을 어떻게 다룰지를 다룬다. 대체물을 쓰기로 한 순간 그 자리는 확인이 지켜 주지 않는 자리가 된다.

3.2 대체물의 종류

대체물은 하는 일에 따라 나뉜다. 가장 단순한 것은 정해진 값을 돌려주기만 하는 것이다. 부르는 쪽이 그 값으로 무엇을 하는지 보는 데 쓴다.

다음은 진짜처럼 동작하되 가볍게 만든 것이다. 저장소 대신 메모리에 값을 담는 대체물이 여기 든다. 여러 번 부르며 상태가 쌓이는 흐름을 확인할 때 쓴다.

마지막은 어떻게 불렀는지 기록해 두는 것이다. 몇 번 불렀는지, 어떤 값으로 불렀는지 확인하는 데 쓴다. 이 종류는 힘이 세지만 구현에 매이기 쉬워, 부르는 것 자체가 기대인 경우에만 쓴다. 알림을 보내야 하는 처리가 그런 경우다. 종류를 섞어 쓰면 확인이 무엇을 겨누는지 흐려지므로 하나를 고른다. 가장 단순한 종류로 되는 확인은 그 종류로 둔다.

3.3 무엇을 대체할까

대체물을 어디에 놓을지는 확인의 성격을 정한다. 너무 많이 대체하면 실제로 도는 코드가 거의 남지 않고, 너무 적게 대체하면 확인이 느려지고 준비가 어려워진다.

기준은 그 구성 요소가 이 확인의 대상인가다. 대상이면 진짜를 쓰고, 대상이 아니면 대체한다. 계산 규칙을 확인하는데 저장소를 진짜로 쓸 이유는 없고, 반대로 저장이 잘 되는지 확인하는데 저장소를 대체하면 확인할 것이 사라진다.

판단이 어려우면 바깥 경계를 기준으로 삼는다. 이 시스템 밖에 있는 것은 대체하고 안에 있는 것은 되도록 진짜로 쓴다. 안쪽까지 대체하기 시작하면 확인이 코드 구조에 매이고, 구조를 바꿀 때마다 확인도 함께 바뀐다. 대체 범위를 적어 두면 그 확인이 지켜 주는 범위도 함께 적히는 셈이다.

3.4 대체물이 주는 착각

대체물은 만든 사람이 생각한 대로만 동작한다. 그래서 대체물로 한 확인은 진짜가 그렇게 동작한다는 가정 위에 서 있다. 이 가정이 틀리면 확인은 통과하는데 실제로는 실패한다.

가장 흔한 어긋남은 실패의 모양이다. 진짜는 시간이 지나 끊기거나 부분만 응답하는데, 대체물은 깔끔하게 실패를 돌려주도록 만들어진다. 그래서 확인은 통과하지만 실제 실패는 다르게 나타난다.

값의 모양도 어긋난다. 진짜가 돌려주는 자료에는 빈 값이나 예상 못 한 항목이 섞이는데 대체물은 깨끗한 값만 돌려준다. 이 어긋남을 줄이려면 대체물을 만들 때 실제 응답을 보고 만들고, 실패의 모양도 실제에서 관찰한 것으로 넣는다. 실제로 한 번도 보지 않고 만든 대체물은 짐작을 확인으로 바꾼 것에 가깝다.

대체물이 놓이는 자리

진짜로 도는 부분이 그 확인이 지켜 주는 부분이다

규칙만 보는 확인



저장까지 붙이는 확인



전체를 부르는 확인



빗금은 대체물을 놓은 자리다

빗금 친 자리는 이 확인이 지켜 주지 않는다

3.6 대체물을 줄이는 설계

대체물이 많이 필요하다는 것은 확인하려는 코드가 바깥에 많이 기대고 있다는 뜻이다. 그래서 대체물을 잘 만드는 것보다 덜 필요하게 만드는 편이 근본적이다.

방법은 계산과 바깥 접근을 나누는 것이다. 값을 받아 판단하고 결과를 돌려주는 부분과, 그 결과로 저장하거나 부르는 부분을 나누면 앞쪽은 대체물 없이 확인할 수 있다. 대개 확인하고 싶은 규칙은 앞쪽에 있다.

이렇게 나누면 대체물이 필요한 자리가 얇아진다. 얇은 자리는 대체물도 단순해지고, 대체물이 실제와 어긋날 여지도 준다. 확인하기 쉬운 코드를 만드는 일이 확인을 잘 짜는 일보다 앞선다는 말이 여기서 나온다. 대체물이 하나도 필요 없는 확인이 가장 오래 살아남는다.

3.7 대체물이 낡을 때

대체물의 가장 조용한 문제는 낡는 것이다. 상대가 응답 모양을 바꾸거나 새 실패를 돌려주기 시작해도 대체물은 예전 그대로 동작한다. 확인은 계속 통과하고 실제만 실패한다.

이 문제는 대체물을 아무리 잘 만들어도 남는다. 대체물은 진짜와 이어져 있지 않기 때문이다. 그래서 대체물에만 기대지 않고 진짜와의 약속이 지켜지는지 따로 확인하는 방법이 필요하다.

그 방법이 다음 장의 계약 검증이다. 양쪽이 같은 약속을 보고 각자 확인하면, 한쪽이 약속을 바꿨을 때 다른 쪽의 확인이 깨진다. 대체물의 한계를 대체물 안에서 풀 수는 없다는 것이 이 절의 결론이다. 낡은 대체물은 실패하지 않으므로 스스로 알려 주지 않는다.

4.1 통합 확인의 자리

통합 확인은 여러 부분을 실제로 붙여 함께 도는지 보는 확인이다. 단위 확인이 각 부분의 기대를 지킨다면 통합 확인은 부분들 사이의 약속을 지킨다.

붙였을 때만 드러나는 문제가 있다. 넘기는 값의 모양이 서로 다르거나, 부르는 순서가 어긋나거나, 설정이 맞지 않는 경우다. 각 부분은 자기 기대를 지키고 있으므로 단위 확인은 모두 통과한다.

그래서 통합 확인은 개수가 아니라 자리로 정한다. 부분들이 만나는 자리, 특히 자료가 형태를 바꿔 넘어가는 자리에 둔다. 모든 조합을 붙여 보려 하면 수가 폭발하고 느려서 돌리지 못하게 된다. 붙이는 자리마다 하나씩 두면 개수는 적고 덮는 범위는 넓어진다. 자리를 고르는 일이 개수를 정하는 일보다 앞선다.

4.2 계약 검증

계약 검증은 두 구성 요소가 주고받기로 한 약속이 양쪽에서 지켜지는지 보는 확인이다. 부르는 쪽은 약속대로 요청을 만드는지 확인하고, 제공하는 쪽은 약속대로 응답을 돌려주는지 확인한다.

이 방식의 값은 둘을 함께 띄우지 않고도 어긋남을 잡는다는 데 있다. 양쪽이 같은 약속 문서를 보고 각자 확인하므로, 한쪽이 약속을 바꾸면 그 즉시 다른 쪽의 확인이 깨진다.

계약에는 요청과 응답의 모양만이 아니라 조건도 담는다. 어떤 값이 필수인지, 실패는 어떤 모양으로 오는지가 함께 있어야 대체물도 그 계약에 맞춰 만들 수 있다. 계약이 없으면 대체물은 만든 사람의 짐작이 된다. 계약이 적혀 있지 않으면 검증할 대상 자체가 없는 셈이다.

4.3 양쪽에서 같은 계약 보기

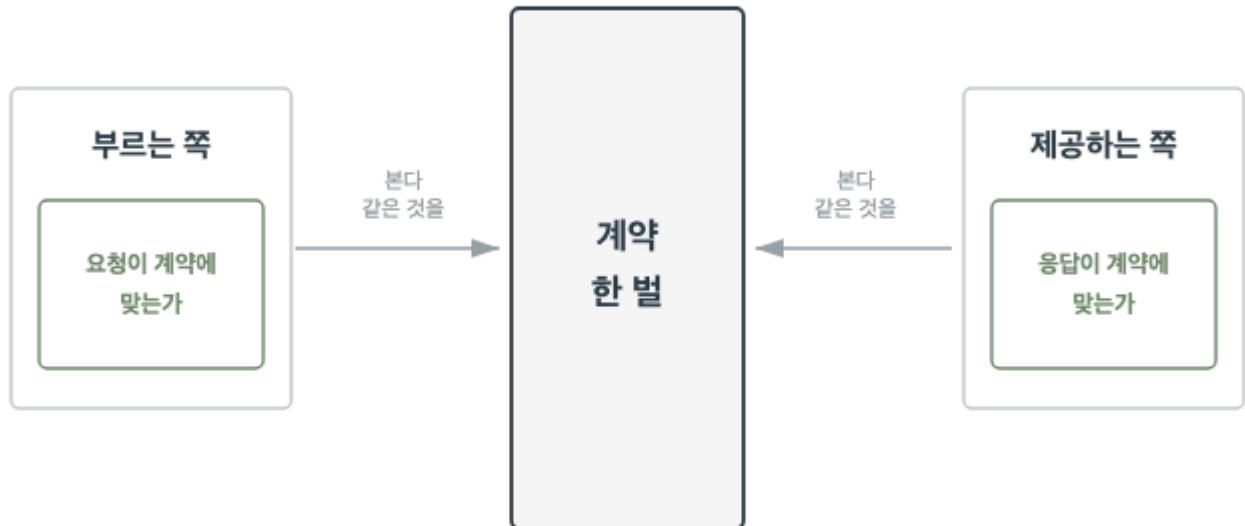
계약 검증이 성립하려면 양쪽이 같은 약속을 보아야 한다. 각자 자기 쪽에 약속을 적어 두면 두 문서가 갈라지고, 갈라진 뒤에는 어느 쪽이 맞는지 판단할 수 없다.

그래서 계약은 한 벌만 두고 양쪽이 그것을 가져다 쓴다. 계약이 바뀌면 양쪽의 확인이 함께 새 계약을 보게 되고, 맞지 않는 쪽에서 확인이 깨진다. 이 깨짐이 계약 검증이 주는 신호다.

계약을 바꿀 때는 앞선 권들에서 다룬 순서를 그대로 따른다. 새 항목을 선택으로 더하고, 양쪽이 옮긴 것을 확인한 뒤, 옛 항목을 지운다. 계약 검증은 이 순서가 지켜지는지 확인해 주는 장치이기도 하다. 계약을 한 벌로 두는 일은 문서 정리가 아니라 검증의 조건이다.

계약을 사이에 둔 두 확인

둘을 함께 띄우지 않고도 어긋남을 잡는다



계약이 바뀌면 양쪽 확인이 함께 깨진다

대체물이 낡는 자리를 여기서 막는다

4.5 통합 확인의 비용

통합 확인은 실제에 가깝지만 비싸다. 준비할 것이 많고, 오래 걸리며, 여러 부분이 함께 도는 만큼 실패했을 때 원인을 다시 좁혀야 한다.

더 큰 비용은 흔들림이다. 붙는 부분이 늘수록 시각이나 순서나 자원 상태에 따라 결과가 달라질 여지가 커진다. 가끔 실패하는 확인은 신뢰를 잃고, 신뢰를 잃은 확인은 실패해도 무시된다.

그래서 통합 확인은 적게 두고 잘 관리한다. 매번 돌리지 않고 합치기 전이나 배포 전에 돌리며, 실패했을 때 원인을 좁힐 수 있도록 준비 상태를 기록으로 남긴다. 통합 확인의 개수를 늘리는 것은 대개 답이 아니다. 비용을 줄이지 못하면 결국 아무도 돌리지 않는 확인이 된다. 적게 두고 잘 관리하는 편이 많이 두고 방치하는 편보다 낫다.

4.6 무엇까지 붙여 볼까

통합 확인에서 어디까지 진짜로 붙일지는 그 확인이 무엇을 지켜 주려는지에 달려 있다. 저장에 잘 되는지 보려면 저장소는 진짜여야 하고, 바깥 연동은 대체해도 된다.

실용적인 기본은 이 시스템 안쪽은 진짜로 붙이고 바깥은 대체하는 것이다. 바깥까지 진짜로 붙이면 상대의 사정에 따라 확인 결과가 달라지고, 확인이 깨졌을 때 이쪽 문제인지 저쪽 문제인지 알 수 없다.

바깥과의 약속은 계약 검증으로 따로 지킨다. 이렇게 나누면 통합 확인은 이쪽 안에서 결정되고, 바깥과의 어긋남은 계약 검증에서 드러난다. 두 장치가 겹치지 않게 맡을 자리를 나누는 것이 이 장의 결론이다. 경계를 정해 두면 확인이 깨졌을 때 볼 자리도 함께 정해진다.

4.7 바깥 시스템과의 확인

바깥 시스템과의 확인에는 이쪽만으로 정할 수 없는 것이 있다. 상대의 시험 환경을 쓸 수 있는지, 그 환경이 실제와 같은지, 언제 쓸 수 있는지가 상대의 사정에 달려 있다.

그래서 대개 세 겹으로 둔다. 계약 검증으로 약속의 모양을 확인하고, 대체물로 이쪽 코드의 흐름을 확인하며, 상대의 시험 환경에는 이따금만 실제로 붙여 본다. 마지막 것은 자주 돌리지 않으므로 실패를 빨리 알아채기 어렵다는 점을 감안한다.

상대가 계약을 지키지 않는 경우도 있다. 이때는 확인으로 막을 수 없으므로 앞선 권들에서 다룬 실패 경계로 다룬다. 확인은 이쪽이 약속을 지키는지 보는 장치이지 상대를 강제하는 장치가 아니다. 세 겹 가운데 무엇이 무엇을 지켜 주는지 적어 두면 빈자리가 보인다.

5.1 회귀의 뜻

회귀는 한 번 되던 것이 나중에 다시 안 되는 일이다. 새 기능을 넣다가 옛 동작이 깨지거나, 고친 버그가 다시 나타나는 경우다.

회귀가 특히 나쁜 이유는 아무도 그 자리를 보고 있지 않기 때문이다. 새로 짰 부분은 확인하지만 예전에 되던 부분은 다시 확인하지 않는다. 그래서 회귀는 대개 사용자가 먼저 발견한다.

확인이 회귀를 막는 방식은 단순하다. 한 번 적어 둔 기대는 계속 확인되므로, 그 기대를 깨는 변경이 들어오면 곧바로 드러난다. 확인의 목록이 쌓일수록 지켜지는 기대가 늘고, 그만큼 손댈 수 있는 범위가 넓어진다. 회귀를 막는 일은 새 기능을 만드는 일보다 눈에 띄지 않지만 더 오래 쓰인다. 회귀를 겪을 때마다 목록에 기대가 하나씩 늘어난다.

5.2 버그에서 확인으로

문제를 고칠 때 순서를 하나 둔다. 고치기 전에 그 문제를 드러내는 확인을 먼저 만든다. 이 확인은 고치기 전에는 실패하고 고친 뒤에는 통과해야 한다.

이 순서가 주는 것은 둘이다. 확인이 실제로 그 문제를 겨누고 있는지 알 수 있고, 같은 문제가 다시 왔을 때 곧바로 걸린다. 고친 뒤에 확인을 만들면 그 확인이 문제를 잡을 수 있는지 확인할 방법이 없다.

확인 은 증상을 기준으로 적는다. 고친 방법을 기준으로 적으면 나중에 다르게 고쳤을 때 그 확인이 함께 의미를 잃는다. 어떤 조건에서 무엇이 나와야 하는지만 적으면 구현이 바뀌어도 살아남는다. 실패하는 것을 한 번 보지 않은 확인은 통과해도 뜻을 알 수 없다.

5.3 확인이 쌓이는 방식

확인 은 한 번에 설계되지 않고 쌓인다. 새 기능마다 늘고, 겪은 문제마다 늘고, 바꾼 계약마다 는다. 쌓이는 것 자체는 정상이다.

문제는 쌓이기만 하고 정리되지 않을 때 생긴다. 같은 것을 여러 층에서 확인하고, 이미 없어진 기능의 확인이 남고, 무엇을 겨누는지 아무도 모르는 확인이 남는다. 그러면 전체를 돌리는 시간이 늘고 실패해도 원인을 알기 어려워진다.

그래서 확인 목록도 코드처럼 다룬다. 더할 때 어느 층에 둘지 정하고, 겹치면 아래층에 남기고 위층에서 빼며, 쓸모가 없어진 것은 지운다. 확인을 지우는 일이 확인을 더하는 일만큼 정상적인 작업이라는 인식이 필요하다. 쌓인 목록을 이따금 훑는 일이 새 확인을 더하는 일만큼 필요하다.

5.4 느려지는 확인

확인이 늘면 도는 시간도 는다. 시간이 늘면 자주 돌리지 않게 되고, 자주 돌리지 않으면 깨진 것을 늦게 알게 된다. 늦게 알수록 원인이 되는 변경이 여러 개 쌓여 좁히기 어려워진다.

느려지는 원인은 대개 몇 개의 확인에 몰려 있다. 전체 시간을 확인별로 나눠 보면 대부분이 소수의 느린 확인에서 나온다. 그 몇 개를 손보는 것이 전체를 조금씩 줄이는 것보다 효과가 크다.

손보는 방법은 셋이다. 층을 낮춰 대체물로 바꾸거나, 준비 과정을 여러 확인이 함께 쓰게 하거나, 확인을 나눠 자주 돌릴 것과 이따금 돌릴 것으로 가르다. 모든 확인을 언제나 돌린다는 규칙을 버리는 것이 대개 첫 단계다. 전체 시간을 확인별로 나눠 보는 일이 줄이기의 첫 단계다.

5.5 흔들리는 확인

코드를 바꾸지 않았는데 어떤 때는 통과하고 어떤 때는 실패하는 확인이 있다. 이런 확인은 신뢰를 무너뜨린다. 실패해도 다시 돌려 보고 넘어가게 되고, 그러다 진짜 실패까지 함께 무시된다.

흔들리는 원인은 대개 정해져 있다. 시각이나 무작위 값에 기대거나, 확인들이 같은 자료를 공유하며 순서에 따라 결과가 달라지거나, 기다리는 시간을 고정해 두어 느린 날에 실패하는 경우다.

다루는 방법은 원인별로 다르지만 원칙은 하나다. 흔들리는 확인은 고치거나 지운다. 그대로 두면 확인 전체의 신뢰가 떨어져 다른 확인의 값까지 함께 잃는다. 잠시 꺼 두기로 했다면 언제 다시 볼지 함께 적는다. 흔들림을 방치하면 실패가 신호가 아니라 소음이 된다.

5.6 확인을 지우는 기준

확인을 지우는 일은 조심스럽지만 필요하다. 기준은 그 확인이 지금도 어떤 기대를 지켜 주는가다. 지켜 주는 기대가 없으면 지운다.

지워도 되는 전형은 셋이다. 없어진 기능의 확인, 아래층에서 이미 같은 것을 확인하는 위층의 확인, 그리고 구현을 그대로 옮겨 적어 고칠 때마다 함께 고치는 확인이다.

지우기 전에 한 가지를 한다. 그 확인이 겨누던 기대가 다른 확인에도 남아 있는지 본다. 남아 있지 않는데 그 기대가 여전히 유효하다면, 지우는 대신 알맞은 층으로 옮긴다. 지우는 판단은 기대 단위로 하고 확인 단위로 하지 않는다. 지운 이유를 한 줄 남기면 나중에 그 기대가 필요해졌을 때 되살릴 수 있다. 지울 확인을 고르는 눈은 더할 확인을 고르는 눈과 같다.

5.7 확인이 알려 주는 설계

확인을 짜다 보면 코드의 구조가 드러난다. 준비할 것이 많으면 의존이 많고, 대체물이 많이 필요하면 바깥에 많이 기대며, 확인이 자주 깨지면 기대가 아니라 구현에 매여 있다.

이 신호를 확인 쪽 문제로만 보면 확인을 억지로 맞추게 된다. 준비 코드를 도구로 감추고, 대체물을 더 정교하게 만들고, 깨질 때마다 고친다. 그러면 신호는 사라지지만 구조는 그대로 남는다.

그래서 확인이 짜기 어려울 때는 먼저 코드를 본다. 대개 함수를 나누거나 바깥 접근을 한 자리로 모으면 확인이 저절로 단순해진다. 확인은 결과를 지키는 장치이면서 구조를 비추는 거울이다. 확인이 쉬워지는 방향과 구조가 나아지는 방향은 대개 같다. 거울을 가리는 대신 비친 모습을 고치는 편이 오래 간다.

6.1 감쌀 기능과 상태

이 장은 확인이 거의 없던 기능 하나를 확인으로 감싼 기록이다. 대상은 주문 취소 처리였다. 취소 가능 여부를 판단하고, 재고를 되돌리고, 환불을 요청하고, 기록을 남기는 네 단계로 이루어져 있었다.

확인은 두 개뿐이었고 둘 다 정상 흐름만 보고 있었다. 취소가 안 되는 조건, 환불 요청이 실패하는 경우, 이미 취소된 주문을 다시 취소하는 경우는 아무도 확인하지 않았다.

손대게 된 계기는 취소 조건을 하나 바꾸라는 요구였다. 지금 동작이 무엇인지 아무도 확신하지 못했고, 고쳤을 때 무엇이 깨질지도 알 수 없었다. 그래서 고치기 전에 먼저 감싸기로 했다.무엇이 기대인지 모르는 상태에서 손대는 것이 가장 비싼 변경이다.

6.2 기대를 먼저 적기

먼저 이 기능이 무엇을 하기로 되어 있는지 적었다. 코드를 읽어 적으면 지금 동작을 그대로 옮기게 되므로, 업무 담당자에게 묻고 화면의 안내 문구를 보고 기대를 모았다.

모은 기대는 열한 개였다. 어떤 조건에서 취소가 되는지, 안 될 때 무엇을 알리는지, 환불 요청이 실패하면 재고를 어떻게 하는지 같은 것들이다. 이 가운데 셋은 아무도 답을 알지 못해 결정이 필요했다.

그다음에 코드를 읽었다. 열한 개 가운데 여덟 개는 코드와 맞았고, 둘은 달랐으며, 하나는 코드에 아예 없었다. 다른 둘은 확인을 만들어 지금 동작이 어느 쪽인지 드러나게 했다.결정이 필요한 셋은 담당자와 정하고 그 결과를 확인으로 옮겼다.

6.3 총을 나눈 기준

열한 개의 기대를 총에 나눴다. 취소 가능 여부 판단처럼 값만으로 답이 나오는 기대는 단위 확인으로 두었다. 여섯 개가 여기 들어갔다.

재고가 실제로 되돌아가는지, 이미 취소된 주문을 다시 취소하면 두 번 되돌아가지 않는지처럼 저장까지 봐야 하는 기대는 통합 확인으로 두었다. 셋이 여기 들어갔다.

환불 요청과 관련된 둘은 바깥 시스템이 걸려 있어 계약 검증과 대체물로 나눴다. 요청의 모양이 약속에 맞는지는 계약 검증이 보고, 요청이 실패했을 때 이쪽이 어떻게 하는지는 대체물로 확인했다. 이 나눔이 이번 작업에서 가장 오래 고민한 부분이다. 나눔이 끝나자 무엇을 확인하지 않고 있는지도 함께 드러났다. 총을 나눈 표는 작업이 끝난 뒤에도 그대로 남겨 두었다.

6.4 대체물을 만든 방식

환불 요청의 대체물은 계약을 보고 만들었다. 성공 응답과 실패 응답의 모양을 계약에서 가져왔고, 실패의 종류도 계약에 적힌 셋을 그대로 넣었다.

여기에 하나를 더 넣었다. 실제 기록을 뒤져 보니 계약에 없는 상황이 하나 있었다. 응답이 아예 오지 않고 시간이 지나 끊기는 경우다. 이 경우를 대체물에 넣자 이쪽 코드가 그 상황을 다루지 못한다는 것이 드러났다.

대체물을 계약만 보고 만들었다면 이 문제를 놓쳤을 것이다. 계약은 상대가 지키기로 한 것이고, 실제 기록은 상대가 실제로 한 것이다. 대체물은 둘을 모두 보고 만들어야 한다는 것을 이 자리에서 배웠다. 계약과 기록을 함께 본 것이 이 대체물을 실제에 가깝게 만들었다.

6.5 통합 확인의 준비

통합 확인 셋은 저장소를 진짜로 붙였다. 준비가 문제였다. 주문 하나를 만들려면 회원과 상품과 재고가 먼저 있어야 했고, 그 준비 코드가 확인 본문보다 길었다.

그래서 준비를 한 자리에 모아 확인들이 함께 쓰게 했다. 기본 상태를 만드는 함수를 하나 두고, 확인마다 다른 부분만 바꾸게 했다. 준비 코드가 짧아지자 각 확인이 무엇을 겨누는지 눈에 들어왔다.

준비가 길었다는 사실 자체도 적어 두었다. 취소 처리가 여러 자료에 걸쳐 있다는 신호이고, 나중에 구조를 볼 때 다시 꺼낼 기록이기 때문이다. 이번에는 구조를 바꾸지 않고 확인만 갖추기로 했다.준비를 공유하자 새 확인을 더하는 비용도 함께 줄었다.준비 함수 하나가 확인 셋의 길이를 절반으로 줄였다.

6.6 확인이 드러낸 것

확인을 갖추는 동안 문제가 넷 나왔다. 이미 취소된 주문을 다시 취소하면 재고가 두 번 되돌아갔고, 환불 요청이 실패해도 취소 상태가 그대로 남았으며, 취소 불가 조건 하나가 화면 안내와 달랐고, 기록에 남는 사유가 비어 있는 경우가 있었다.

넷 다 고치기 전에 확인을 먼저 만들어 실패하는 것을 보고 고쳤다. 고친 뒤 확인이 통과하는 것까지 보고 나서야 다음으로 넘어갔다.

이 넷은 모두 예전부터 있던 문제였다. 사용자가 알리지 않았거나 알렸는데 재현되지 않아 넘어간 것들로 보인다. 확인을 갖추는 일이 새 결함을 만들지 않으면서도 있던 결함을 드러낸다는 것을 이 작업에서 확인했다. 넷 다 확인이 없었다면 다음 변경에서 함께 터졌을 문제들이다.

6.7 원래 하려던 변경

감싸는 일이 끝난 뒤 원래 요구했던 취소 조건 변경을 했다. 조건 하나를 바꾸자 확인 둘이 깨졌다. 하나는 바뀌어야 하는 기대라 확인을 함께 고쳤고, 다른 하나는 바뀌면 안 되는 기대라 코드를 고쳤다.

깨진 확인이 둘 다 뜻이 있었다는 점이 중요하다. 감싸기 전이었다면 두 번째 문제는 배포된 뒤에 드러났을 것이다. 변경에 걸린 시간은 한 시간이 되지 않았다.

감싸는 데는 사흘이 걸렸다. 한 번의 변경만 보면 비용이 크지만, 이 기능은 이후로도 여러 번 바뀌었다. 확인을 갖추는 비용은 첫 변경이 아니라 이어질 변경들과 견주어야 한다는 것이 이 기록의 결론이다. 감싸기의 값은 이번 변경이 아니라 다음 변경에서 드러난다.

7.1 얼마나 확인할 것인가

확인 = 많을수록 좋은 것이 아니다. 유지 비용이 있고, 도는 시간이 있고, 잘못 짜인 확인은 변경을 막는다. 그래서 얼마나 확인할지도 결정이다.

판단 기준은 그 기대가 깨졌을 때의 손해와 그 확인을 유지하는 비용이다. 돈이나 권한이 걸린 기대는 손해가 크므로 넉넉히 확인하고, 겉모습처럼 자주 바뀌고 손해가 작은 것은 적게 확인한다.

뒤편 비율을 목표로 삼는 방식은 이 판단을 대신해 주지 못한다. 비율은 어떤 줄이 실행되었는지 알려 줄 뿐 그 줄의 기대가 확인되었는지 알려 주지 않는다. 비율을 올리려고 짠 확인은 대개 아무 기대도 지키지 않는다. 확인의 양을 정하는 일도 설계이므로 근거를 적어 둔다. 무엇을 확인할지 정하는 일이 얼마나 확인할지 정하는 일보다 앞선다.

7.2 통과를 어떻게 읽을까

확인이 모두 통과했다는 말은 적어 둔 기대가 지켜졌다는 뜻이지 코드에 문제가 없다는 뜻이 아니다. 그래서 통과는 배포해도 된다는 판단의 재료 가운데 하나일 뿐이다.

함께 볼 것은 무엇이 확인되지 않았는지도. 이번 변경이 건드린 자리 가운데 확인이 없는 자리가 있다면 그 자리가 이번 배포의 위험이다. 확인 결과를 볼 때 통과한 목록만 보고 확인되지 않은 자리를 보지 않으면 판단이 한쪽으로 기운다.

반대로 실패는 분명한 신호다. 실패한 확인을 끄거나 무시하고 배포하는 습관이 들면 확인 전체가 값을 잃는다. 실패를 다루는 방법은 고치거나, 기대가 바뀌었다면 확인을 고치거나, 그 확인을 지우는 것 셋뿐이다. 확인 결과는 판단의 재료이지 판단 자체가 아니다.

7.3 총을 다시 배분할 때

총 구성은 한 번 정하면 그대로 가지 않는다. 통합 확인이 늘어 도는 시간이 길어지거나, 단위 확인이 구현에 매여 자주 깨지면 배분을 다시 본다.

다시 볼 때 기준은 각 기대가 가장 싼 총에서 확인되고 있는가다. 값만으로 답이 나오는 기대가 통합 확인에 있으면 아래로 내리고, 붙였을 때만 드러나는 기대가 단위 확인에서 대체물로 흉내 내지고 있으면 위로 올린다.

옮기는 일은 한 번에 하나씩 한다. 옮기고 나서 전체 시간과 깨지는 빈도를 견주면 그 판단이 맞았는지 알 수 있다. 총 배분은 정답이 아니라 지금 상황에 맞춘 배치이므로 상황이 바뀌면 다시 옮긴다. 배분을 적어 두지 않으면 옮길 근거도 남지 않는다. 옮긴 뒤의 수치를 견주지 않으면 배분이 나아졌는지 알 수 없다.

7.4 확인 전략 점검 항목

확인 전략을 점검할 때는 다음을 본다. 첫째, 각 기대가 어느 층에서 확인되는지 말할 수 있는가. 말할 수 없으면 배분이 없는 것이다.

둘째, 자주 돌리는 확인이 몇 분 안에 끝나는가. 셋째, 바깥과 주고받는 약속이 계약으로 적혀 있고 양쪽이 같은 것을 보는가. 넷째, 대체물이 실제 응답과 실패를 근거로 만들어졌는가.

다섯째, 흔들리는 확인이 있는가. 있다면 그 하나가 나머지 전체의 신뢰를 갉아먹는다. 걸리는 항목이 있으면 그 주제를 다룬 앞 절을 다시 본다. 다섯 가운데 셋째와 넷째가 가장 자주 빠진다. 다섯 항목은 새 기능을 시작할 때 그대로 다시 쓰인다. 점검은 확인을 더하기 전에 하는 편이 순서가 맞다. 빠지기 쉬운 항목일수록 목록에 적어 두는 값이 크다.

7.5 확인을 고치는 순서

확인이 이미 쌓여 있는 상태에서 손볼 때는 순서가 있다. 가장 먼저 흔들리는 확인을 다룬다. 신뢰가 없으면 다른 개선의 결과도 읽을 수 없기 때문이다.

다음은 도는 시간이다. 자주 돌리는 묶음이 몇 분 안에 끝나게 만들면 확인이 실제로 쓰이기 시작한다. 그다음은 총 배분이고, 마지막이 빠진 기대를 채우는 일이다.

순서를 이렇게 두는 이유는 앞의 것이 뒤의 것의 조건이기 때문이다. 신뢰할 수 없고 느린 확인 묶음에 새 확인을 더하면 문제만 커진다. 확인을 늘리기 전에 있는 확인이 실제로 쓰이는 상태인지 먼저 만든다. 순서를 지키면 앞 단계의 개선이 뒤 단계의 판단을 쉽게 만든다. 네 단계를 한 번에 하려 하면 어느 것도 끝나지 않는다.

7.6 확인과 다른 안전 장치

확인은 안전 장치의 하나일 뿐이다. 사람이 읽는 검토가 있고, 실행 중에 값을 막는 검증이 있고, 실패를 가두는 경계가 있고, 배포 뒤에 지켜보는 관찰이 있다.

확인이 잘하는 것은 적어 둔 기대가 계속 지켜지는지 보는 일이다. 잘 못하는 것은 생각하지 못한 상황을 잡는 일이다. 그래서 확인으로 모든 것을 막으려 하면 수만 늘고 빠진 자리는 그대로 남는다.

생각하지 못한 상황은 실행 중의 검증과 경계와 관찰이 맡는다. 확인은 아는 것을 지키고 나머지 장치는 모르는 것을 다룬다. 이 나눔이 서 있으면 확인을 어디까지 만들지 판단하기 쉬워진다. 장치마다 맡을 자리를 적어 두면 어느 자리가 비었는지 보인다. 확인만으로 안전을 만들려 하면 확인의 수가 끝없이 늘어난다.

7.7 핵심 원칙 정리

이 책이 반복한 물음은 둘이다. 이 확인은 어떤 기대를 지켜 주는가, 그리고 그 기대는 어느 층에서 가장 싸게 지켜지는가. 앞의 물음이 확인의 내용을 정하고 뒤의 물음이 배치를 정한다.

나머지는 두 물음의 도구다. 단위 확인은 좁은 기대를 빠르게 지키고, 대체물은 만들 수 없는 상황을 만들어 주며, 계약 검증은 대체물이 낡는 자리를 막고, 회귀 방지는 한 번 적은 기대를 계속 지킨다.

끝으로 덧붙일 것은 확인의 값이 개수에 있지 않다는 점이다. 무엇을 지켜 주는지 말할 수 있는 확인 하나가 아무도 목적을 모르는 확인 열보다 낫다. 확인을 늘리기 전에 지금 있는 확인이 무엇을 지키고 있는지 말할 수 있는지 먼저 묻는다. 무엇을 지키는지 말할 수 없는 확인은 지금 지워도 잃을 것이 없다.